

SwiftUI Flashcard App

A simple, interactive flashcard application built entirely with SwiftUI for iOS. It demonstrates swipe gestures, animations, and Core Data integration for managing flashcards.

Description

This app provides a clean interface for studying flashcards. Users can view cards one by one, tap to flip between the question and answer, and swipe left or right to move to the next card. The app persists the flashcards using Core Data.

Motivation for Creating

This project was undertaken to deepen understanding and proficiency in building modern iOS applications using SwiftUI. Key goals included:

- Exploring SwiftUI's capabilities for creating fluid user interfaces and complex animations (like card flips and swipes).
- Gaining practical experience with Core Data for local data persistence within a SwiftUI context.
- Implementing common mobile UI patterns, such as gesture recognition and dynamic view updates.
- Creating a functional and useful tool for personal learning or demonstration purposes.

Features

- **Swipeable Card Interface:** Navigate through flashcards using intuitive swipe gestures.
- **Flip Animation:** Tap a card to smoothly flip it and reveal the answer (or question).
- **Smart Swipe Animation:**
 - Cards swipe off-screen if dragged beyond a certain threshold.
 - Cards snap back to the center with a spring animation if the drag is short.
- **Smooth Transitions:** The next card is subtly visible beneath the current one and animates smoothly into place (scaling up) after a swipe.
- **Core Data Persistence:** Flashcards are saved locally using Core Data, ensuring your data persists across app launches.
- **Card Management:** Includes a dedicated screen (accessed via the "Manage" button) to add, view, (and potentially edit/delete - depending on `ManagerView` implementation) flashcards.
- **Built with SwiftUI:** Leverages modern SwiftUI features for UI, state management, gestures, and animations.

Technology Stack

- **UI Framework:** SwiftUI
- **Persistence:** Core Data
- **Language:** Swift
- **Platform:** iOS

Code Highlights

- **ContentView.swift :**
 - Manages the main UI, including the `ZStack` for layering cards.
 - Uses `@FetchRequest` to load `FlashCard` entities from Core Data.
 - Implements the `DragGesture` to handle card dragging and swiping logic.
 - Uses `withAnimation` and `DispatchQueue.main.asyncAfter` to coordinate complex animations (swipe-off, snap-back, next card scale-up).
 - Calculates swipe thresholds based on card geometry.
- **CardView.swift :**
 - Represents a single flashcard.
 - Handles the tap gesture and `rotation3DEffect` for the flip animation.
 - Uses `.id()` and `.onChange()` to ensure the flip state resets correctly when the card data changes.
- **Core Data Integration:**
 - `FlashCard` entity (defined in your `.xcdatamodeld` file).
 - `@Environment(\.managedObjectContext)` is used to pass the Core Data context down the view hierarchy, especially to the `ManagerView`.
 - `PersistenceController.swift` likely contains the boilerplate Core Data stack setup.
- **ManagerView.swift :** (Implementation details assumed)
 - Likely displays a list of existing flashcards.
 - Provides functionality to add new `FlashCard` entities to the Core Data store.
 - (May include editing and deletion capabilities).

Future Improvements

- Group cards into Decks or Categories.
- Add search/filtering capabilities to the `ManagerView`.
- Implement different study algorithms (e.g., Spaced Repetition).
- Add iCloud syncing for flashcards across devices.
- Write Unit and UI Tests.